SpringGuard 技术报告

SpringGuard：基于 Open Code Review 的 Spring 智能审查增强

项	内容
作品名称	SpringGuard 智能审查增强
参赛方向	AI 开发工具与开源协作
作品形态	开发工具 / 二次开发增强（Web + CLI + 规则包 + 评测）
成果仓库	https://github.com/NEX-coding/springguard
日期	2026 年 10 月

> 正文不含校名、校徽与指导教师信息。结构对照《AIC·AI+开源技术报告参考大纲》，建议 ≤15 页。

一、作品概述

1.1 背景与问题

Spring Boot 是校园实训与中小团队后端的主流技术栈。Code Review 与线上故障中反复出现：

1. `@Transactional` 同类 `this.xxx()` **自调用**导致代理失效；
2. Controller `@RequestBody` **缺少** `@Valid` / `@Validated`；
3. `Map.get` 等可空结果直接调用方法引发 **NPE**；
4. 字符串拼接或 MyBatis 注解误用 **`${}`** 带来注入风险；
5. 事务方法中 **吞掉异常**，破坏回滚语义。

纯大模型审查存在漏报、位置漂移、不可复核；纯人工成本高。需要**可演示、可复现、开源边界清晰**的 Spring 场景审查增强方案。

1.2 目标、方向与形态

- **目标**：无大模型时仍能稳定检出核心缺陷；可挂接 Open Code Review（OCR）与本地 Ollama；提供金标准评测（Recall / Precision）；提供可上传 ZIP 的 Web 服务与 CLI。
- **参赛方向**：**AI 开发工具与开源协作**。

- **形态**：在合法开源审查底座上的领域增强，而非从零重写通用审查引擎，也非薄封装部署。

1.3 核心亮点

1. **自研 SpringGuard**：Controller / Service / Mapper 确定性规则，秒级可复现。
2. **开源融合**：不修改 OCR 源码；通过 `.opencodereview/rule.json` 与外部编排调用上游。
3. **可验证**：正例 `spring-buggy`（6 类缺陷）+ 负例 `spring-clean`；本轮 **Recall = 1.0, Precision = 1.0**。
4. **可交付体验**：Web 可上传工程 ZIP / 扫官方样例 / 跑金标准评测；成果已公开于 GitHub。

二、需求分析与总体方案

2.1 需求分析

优先级	需求	说明
P0	稳定检出 Spring 高发缺陷	演示与落地底线
P0	无 API Key 可跑通	机房 / 答辩友好
P1	挂接 OCR + 本地模型	体现 AI+开源融合
P1	金标准 + 负例评测	可验证效果
P1	Web 上传与结果可视化	他人可直接使用

2.2 总体方案

```
用户 ZIP / samples/spring-buggy|spring-clean
        │
        ├─► SpringGuard（自研静态规则）──► findings
        │
        └─► Open Code Review（上游，可选）
                + .opencodereview/rule.json
                ──► OCR findings
        │
        └─► merge → eval（Recall / Precision）→ Web（:5193）
```

2.3 可行性与成本

演示以确定性扫描为主（秒级）；OCR 可选。软件栈为 Python 3 + 可选 Node OCR CLI + 可选 Ollama，符合学生团队周期与算力。

三、AI 技术与开源及第三方资源选择

3.1 AI 技术作用

环节	作用	如何验证
OCR Agent（可选）	语义级审查与解释	`ocr llm test`；与金标准对比
Spring 规则包	引导 Agent 关注 Spring 模式	规则 JSON 可审计
SpringGuard	**非生成式**底线检出	`eval/run_eval.py`

为何需要 AI：跨文件与变体命名适合 Agent。为何不能只靠 AI：比赛要求可复核，故 Guard 为主、OCR 为增强。

3.2 开源及第三方资源

详见附件《开源及第三方资源使用清单》。核心：Open Code Review（Apache-2.0，未改源码）；Ollama / 模型权重可选且不随作品分发；Python 标准库承载自研引擎。

3.3 自主开发与权利边界

归属	内容
上游	OCR 实现与默认引擎
自研	`spring_guard.py`、`app_server.py`、规则包、流水线、金标准、正/负例样例、Web、文档
AI 辅助	可用于辅助编写；规则与金标准经团队设计、运行与人工复核

3.4 融合方式（非薄封装）

融合 = 上游扩展点（规则 JSON、CLI）+ 自研领域引擎 + 评测闭环 + Web 交付。相对「只跑 OCR」增加可复现 Spring 规则与 Precision；相对「只写脚本」保留开源 AI 审查挂接能力。

四、设计与实现过程

4.1 核心功能

1. CLI：`spring_guard.py` / `run_pipeline.py`
2. Web：`app_server.py` — 扫样例、上传 ZIP（≤50MB）、金标准评测
3. 评测：`eval/run_eval.py` — Recall + Precision（含 clean）
4. 演示：`./scripts/demo.sh` → http://127.0.0.1:5193/

4.2 关键实现

- 路径启发式识别 Controller / Service / Mapper。
- 规则：缺校验、NPE 链路（有判空则抑制）、事务自调用、吞异常、SQL 拼接、`${}`。

- 注释剥离，避免注释中的 `@Valid` 干扰。
- 上传 ZIP 自动定位含 `src/main/java` 的工程根。
- 演示默认可跳过 OCR，保证答辩可复现。

4.3 AI 工具使用说明

开发中可使用生成式 AI 辅助脚本与文档；纳入作品的代码与评测均经本地运行验证。不以模型输出作为「已检出」的唯一依据。

五、成果展示与效果验证

5.1 成果展示

- 公开仓库：https://github.com/NEX-coding/springguard
- 一键演示：`./scripts/demo.sh`
- Web：上传 ZIP、严重级别筛选、缺陷详情与证据代码

5.2 验证方法

集合	路径	作用
正例	`samples/spring-buggy`	Recall
负例	`samples/spring-clean`	误报 / Precision
金标准	`eval/gold.json`（6 类）	命中判定

5.3 验证结果（可脚本复现）

模式 / 集合	Findings	TP	Recall	Precision
spring_guard（buggy）	7	6/6	**1.00**	**1.00**
spring-clean	**0**	—	—	**1.00**（无误报）
ocr_only（本轮未完成长时 scan）	0	0	0.00	—
hybrid（本轮≈Guard）	7	6/6	**1.00**	1.00

```
python3 scripts/run_pipeline.py --skip-ocr
python3 eval/run_eval.py
```

已达成：六类缺陷全召回；干净工程 0 误报；Web/CLI 可演示。
待加强：更强算力下补全 ocr_only 对比；扩大真实负例集。

六、创新性与开放价值

6.1 创新性

相对裸 OCR：Spring 高发模式固化为确定性规则。
相对通用静态分析：保留开源 Agent 挂接与领域规则包。
相对薄封装：自研引擎、评测协议、正负例与 Web 构成实质改进。

6.2 开放成果

源码、规则包、评测、样例、Web、文档已公开于 GitHub（https://github.com/NEX-coding/springguard）。建议许可证与仓库 LICENSE 保持一致。

6.3 复用方式

他人可复用 `spring_guard.py` + `rule.json` + `eval/` 协议；或直接运行 `app_server.py` 提供局域网审查服务。

6.4 维护计划

扩大负例；CI 接入；补充 `@Async` 等规则；视情况向上游提交规则示例。

七、合规、安全与风险说明

7.1 许可合规

OCR：Apache-2.0，未改源码，履行署名 / NOTICE。Ollama：MIT（软件）。模型权重：以模型卡为准，**不随作品分发**。

7.2 数据与隐私

样例为团队构造演示代码，无真实个人信息与生产密钥。用户上传工程仅本地临时解压扫描，不得擅自公开扫描结果。

7.3 AI 输出与内容安全

Agent 可能幻觉；以 Guard 与金标准抑制误用。审查结果供人工确认，不自动改生产库。

7.4 公开边界

材料不含校名、校徽、指导教师信息。自主部分权利归依法享有主体；上游权利不变。

7.5 风险与边界

规则为启发式，极端写法可能漏报/误报；面向可读 Java/Spring 源码；OCR 长推理可能超时。

八、总结与展望

本作品完成「开源审查底座 + Spring 确定性增强 + 正负例可复现评测 + Web/CLI 交付」，验证了短周期内实质二次开发的可行性。后续将补全 OCR 对比实验并扩展规则库。

九、附录

附录 A 开源及第三方资源使用清单

见同目录 PDF / Markdown《开源及第三方资源使用清单》。

附录 B 运行与材料

- README.md、`eval/results.json`、`docs/EVAL_REPORT.md`
- 演示视频：`docs/DEMO_VIDEO.mp4`
- 答辩 PPT：`docs/PPT.pdf`

附录 C 作品简介（≤300 字）

Spring 应用中事务自调用、入参未校验、MyBatis 动态 SQL 注入等问题高发，通用大模型审查易漏报且难复核。本作品基于阿里开源 Open Code Review，增加面向 Spring 的确定性规则引擎 SpringGuard 与项目级规则包，形成「确定性检测 + AI 深度审查」混合流水线；在六类已知缺陷样例上 Recall=1.0，干净对照工程 0 误报，结果可脚本复现。相对直接部署上游工具，本方案明确自研边界，突出开源底座与团队增强的融合，并提供可上传工程的 Web 服务，适合演示与校园工程落地。